

SIMATS ENGINEERING
ASSESSMENT TOOL 2: Error Analysis Task

COURSE NAME: Cloud Computing and Big Data Analytics **COURSE CODE:** CSA1522

Register Number	192525435
Name	M. Meghana

COURSE OUTCOME COVERED

CO5: Design big data processing systems using the Hadoop ecosystem including HDFS, MapReduce, and YARN. (BL5)

Dave's Taxonomy: Articulation (Level 4)
Question Count: 5

Total Marks: 25

Code of Conduct

I **M. Meghana (192525435)** certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: *M. Meghana*

Assessment Tasks

Error Analysis Task

1. A Hadoop job repeatedly fails because input splits are uneven and reducers remain idle. Identify the likely design error and propose corrective actions.
2. An HDFS deployment shows frequent data unavailability after a single node failure. Analyze the probable replication or NameNode design error.
3. A data ingestion workflow using ETL and Apache NiFi creates duplicate records in the analytics store. Identify the likely causes and recommend fixes.
4. A YARN cluster suffers from resource starvation when multiple users submit jobs together. Analyze the scheduling error and suggest a better resource management approach.
5. A backup strategy for distributed storage restores outdated data after recovery. Identify the probable design flaw in the replication or backup approach.

Error Analysis Task Rubric

Criteria	Weightage	Level 4 - Exemplary	Level 3 - Proficient	Level 2 - Developing	Level 1 - Beginning
Identification of Error	30%	Accurately identifies the root technical issue	Identifies most of the issue	Partially identifies issue	Fails to identify issue
Cause Analysis	20%	Explains root cause with strong technical reasoning	Reasonable cause analysis	Basic cause analysis	Weak analysis
Corrective Action	25%	Proposes effective and feasible corrections	Reasonable corrections	Basic corrections	Ineffective corrections
Use of Hadoop / Integration Concepts	15%	Applies relevant concepts appropriately	Mostly appropriate application	Partial application	Incorrect application
Clarity	10%	Clear and direct explanation	Generally clear	Somewhat unclear	Poorly presented

1.A Hadoop job repeatedly fails because input splits are uneven and reducers remain idle. Identify the likely design error and propose corrective actions.

Hadoop Job – Uneven Input Splits and Idle Reducers

Problem:

A Hadoop job repeatedly fails because the input splits are uneven and some reducers remain idle.

Error Analysis:

The likely design error is improper data partitioning or input split configuration. If some input splits contain much more data than others, a few map tasks take longer to complete. Similarly, if the partitioner sends a large number of keys to one reducer, it creates data skew. Other reducers finish early and remain idle.

Corrective Actions:

1. Configure an appropriate HDFS block and input split size.
2. Analyze the input data for skewed keys.
3. Use a suitable custom partitioner if required.
4. Increase or properly configure the number of reducers.
5. Use a combiner to reduce intermediate data.
6. Balance the distribution of keys among reducers.
7. Monitor mapper and reducer execution times.

2.An HDFS deployment shows frequent data unavailability after a single node failure. Analyze the probable replication or Name Node design error.

HDFS – Data Unavailability After Node Failure

Problem:

An HDFS deployment experiences frequent data unavailability whenever a single DataNode fails.

Error Analysis:

The main problem is likely **an** insufficient replication factor or poor replica placement. If important data blocks have only one copy, failure of that Data Node makes the blocks unavailable. Another possible problem is that the system has only one Name Node, creating a single point of failure for HDFS metadata.

Corrective Actions:

1. Set the HDFS replication factor to an appropriate value, commonly **3**.
2. Store replicas on different Data Nodes.
3. Enable rack awareness so replicas are distributed across racks.
4. Configure Name Node High Availability with Active and Standby Name Nodes.
5. Monitor under-replicated and missing blocks.
6. Ensure Data Nodes send regular heartbeats to the Name Node.
7. Periodically check cluster health using HDFS monitoring tools.

3.A data ingestion workflow using ETL and Apache NiFi creates duplicate records in the analytics store. Identify the likely causes and recommend fixes.

Apache NiFi – Duplicate Records in Analytics Store

Problem:

An ETL workflow using Apache NiFi creates duplicate records in the analytics database.

Error Analysis:

The likely causes are repeated file processing, retries, network failures, or lack of duplicate detection. If a database insertion succeeds but NiFi does not receive the correct acknowledgment, the same record may be processed again. Poor workflow design can therefore create duplicate records.

Corrective Actions:

1. Use NiFi's **DetectDuplicate** processor.
2. Generate a unique ID for every incoming record.
3. Configure proper retry and failure relationships.
4. Prevent already processed files from being consumed again.
5. Use database primary keys or unique constraints.
6. Design the ETL process to be **idempotent**, meaning repeated processing produces the same final result.
7. Maintain transaction or processing logs.
8. Monitor failed and retried FlowFiles.

Example:

If transaction T101 is received twice, the system should recognize that T101 already exists and avoid inserting it again.

4. A YARN cluster suffers from resource starvation when multiple users submit jobs together. Analyze the scheduling error and suggest a better resource management approach

YARN – Resource Starvation

Problem:

A YARN cluster experiences resource starvation when several users submit jobs at the same time.

Error Analysis:

The probable design error is poor resource scheduling and queue configuration. If one user's application consumes most of the available CPU and memory, other applications have to wait for resources.

Corrective Actions:

1. Use Capacity Scheduler or Fair Scheduler.
2. Create separate queues for different users or departments.
3. Allocate minimum resources to important queues.
4. Set maximum resource limits for each queue.
5. Configure CPU and memory limits.
6. Give priority to critical applications when required.
7. Monitor Resource Manager and Node Manager utilization.
8. Increase cluster resources if the workload consistently exceeds capacity.

Example:

A cluster can have separate queues such as:

- Finance
- Analytics
- Research
- General Users

Each queue receives a controlled percentage of cluster resources.

5.A backup strategy for distributed storage restores outdated data after recovery. Identify the probable design flaw in the replication or backup approach.

Distributed Storage – Outdated Backup After Recovery

Problem:

After a storage failure, the backup system restores an older version of the data instead of the latest valid data.

Error Analysis:

The probable design flaw is an out dated backup or poorly synchronized replication system. A replica may not contain the most recent changes if replication is delayed. Similarly, if backups are performed infrequently, recent transactions may be missing.

Corrective Actions:

1. Perform backups at regular intervals.
2. Use both full and incremental backups.
3. Maintain multiple backup versions.
4. Use snapshots for point-in-time recovery.
5. Monitor replication synchronization.
6. Verify backup completion and integrity.
7. Test restoration periodically.
8. Follow a **3-2-1 backup strategy**:
 - 3 copies of data
 - 2 different storage types
 - 1 copy stored off-site

Example:

If the latest backup was taken at 10:00 AM and the system failed at 3:00 PM, all changes between 10:00 AM and 3:00 PM may be lost. Frequent incremental backups reduce this recovery gap.